On-device Visual Perception for Lightweight Browser Agents

SIH Problem Statement #26171 — ISRO / Dept. of Space

DETAILED MASTER DOCUMENT — v2 (Deep Reference Edition)

This is the single source-of-truth, deep-reference document for the team. It is written to be read by two audiences at once: (1) YOU, so every concept is explained from first principles with no assumed background, and (2) an AI coding agent / any developer picking up a module cold, so every component has an unambiguous spec, exact data shapes, and example code. Read Sections 1-4 fully before writing any code — they remove the ambiguity that causes rework mid-hackathon.

Field	Detail
Problem Statement ID	26171
Organization	Indian Space Research Organisation (ISRO), Dept. of Space
Category / Theme	Software / Smart Automation
Team Size	5 developers
Build Window	3 days to a working prototype + live demo
Doc Purpose	One-stop reference for concepts, architecture, schemas, risks, USPs, and role division

One-line pitch: A privacy filter that sits between your browser and a cloud/offline AI — so an AI agent can help you complete tasks on-screen without ever seeing your private information directly.

Table of Contents

1. The Problem — What, Why, and For Whom

1.1 The everyday problem, explained simply

Imagine an AI assistant that can use your browser for you — filling forms, clicking buttons, completing multi-step tasks — like a helpful robot operating your screen on your behalf. To do this well, that AI needs to SEE your screen and understand what is on it, then decide what to click or type next.

The smartest AI models capable of this kind of visual reasoning (GPT-4-Vision-class models) run on powerful cloud servers, not on your laptop. So normally, to get their help, you would have to send a screenshot of your screen to the cloud.

The catch: your screen usually contains private information — passwords, bank card numbers, your name, someone's face on a video call, internal company data. You do not want to just ship all of that to a third-party server every time you want AI help.

The entire problem statement reduces to one question: **"How do we let a powerful cloud AI help us with what's on our screen, WITHOUT ever showing it our private data?"**

1.2 The solution shape (fixed by the problem statement)

Two cooperating halves:

Part	Job	Runs where
Client (browser extension)	Look at the screen, find anything private, hide it BEFORE anything is sent anywhere, and execute whatever action instruction comes back	User's own laptop — nothing sensitive ever leaves this boundary
Server (backend + LLM)	Receive ONLY the sanitized version, reason about the task, decide the next UI action, send that decision back	Cloud (for the hackathon demo) or fully offline/self-hosted (for a real deployment)

1.3 Why ISRO specifically cares (the real motivation behind the statement)

ISRO is not asking you to help people book movie tickets privately. Think about who actually works at ISRO and what is on their screens day to day:

- Engineers working with satellite telemetry, orbital data, and mission-critical dashboards — often sensitive or classified in nature.
- Ground-station operators using legacy or specialized software where AI-assisted automation (auto-filling forms, navigating complex consoles) would save real time.
- Contractors, interns, and partner-agency staff who would benefit from AI tool assistance but absolutely cannot have screen data leave the premises or device — no government or defense-adjacent organization can casually pipe screenshots of internal systems to a public cloud AI API.
- Multi-vendor, legacy software ecosystems where retrofitting AI-agent capability WITHOUT rewriting the underlying software is valuable — a browser overlay agent is a realistic, non-invasive way to add AI assistance to old systems.

So the underlying real need is: **"We want our engineers to get AI copilot-style help with everyday screen-based tasks, but we cannot risk any sensitive operational, technical, or personnel data leaving our network to reach a cloud LLM."** This is the exact same problem every bank, hospital, and defense org is quietly wrestling with as they try to adopt AI copilots without violating data-governance rules. ISRO is using the browser as a controlled, inspectable sandbox to explore a solution pattern that generalizes far beyond browsers.

1.4 What the evaluation weights are quietly telling you

Signal in the spec	What it implies
Redaction precision (20%) is nearly as heavily weighted as visual accuracy (25%)	Privacy correctness is NOT a secondary feature bolted onto a smart agent — it is co-equal with the agent's intelligence. Do not under-invest in redaction logic to spend more time on the 'cooler' vision/agent part.
Client-side resource utilization (20%) is unusually high for a hackathon rubric	They expect this to plausibly run on modest/older government workstations, not high-end dev laptops with dedicated GPUs. A heavy, 'impressive' local model that saturates the CPU will actively hurt your score even if it's more accurate.
'Any offline deployable (open-source/open-weights) model' explicitly allowed on server	They want to see the pipeline CAN work fully offline / self-hosted, not just 'call a public cloud API and be done.' A cloud API is fine as a hackathon stand-in, but you should be able to say 'and here is how this becomes fully air-gapped in production.'
'DOM tags or any other method' left deliberately open for sanitization	There is no known reference solution. They are exploring this design space through your submissions, not grading you against a fixed 'correct' architecture. Genuine, well-reasoned design choices are rewarded over guessing 'the intended answer.'

1.5 What they are explicitly NOT expecting (do not over-engineer)

- No satellite-specific domain knowledge or orbital-mechanics integration — the task is domain-agnostic; any browser task works as a demo.
- No production-grade, unbreakable security — this genuinely cannot be solved completely in 3 days (see Section 8, 'server-side trust assumption'). A credible prototype plus honest reasoning about trade-offs is the expected bar.
- No training a custom model from scratch — using pretrained lightweight models (Transformers.js models, existing face/PII detectors) is explicitly fine and expected given the timeline.

2. Foundational Concepts Explained

If any term below is unfamiliar to anyone on the team, read this section fully before touching code. Every concept here is used later in the doc without re-explanation.

Browser Extension / Manifest V3

CONCEPT — Browser Extension: A small program you install into Chrome/Firefox that can read and modify web pages you visit, run background scripts, and show its own UI (popups, side panels). Manifest V3 is the current standard format Chrome (and, with small tweaks, Firefox) requires — it defines what permissions the extension needs and which scripts run where.

Content Script vs Background Script

CONCEPT — Content Script: JavaScript code the extension injects directly into a webpage you're viewing. It can read and modify that page's DOM (see below) and is how we detect fields, read the screen structure, and execute actions like clicks.

CONCEPT — Background Script (Service Worker in MV3): Code that runs separately from any specific webpage — handles things like coordinating between tabs, managing extension state, and talking to the server.

DOM (Document Object Model)

CONCEPT — DOM: The structured, tree-shaped representation of a webpage that the browser builds from its HTML — e.g. a page is a tree of elements like , , . Reading the DOM gives you STRUCTURE (this is a password field, this is a submit button) without needing to visually 'look' at pixels at all. This is a free, fast, and highly reliable signal source.

ONNX / ONNX Runtime Web

CONCEPT — ONNX (Open Neural Network Exchange): A universal file format for trained ML models, so a model trained in PyTorch or TensorFlow can be exported once and then run anywhere an ONNX 'runtime' exists — including inside a browser tab. ONNX Runtime Web is the browser-compatible engine that executes these .onnx model files using either WebAssembly (CPU, universally supported) or WebGPU (GPU-accelerated, faster, but not supported on every device/browser yet).

Transformers.js

CONCEPT — Transformers.js: A JavaScript library (from Hugging Face) that lets you load and run pretrained ML models directly in the browser via ONNX Runtime, using a simple 'pipeline' API — e.g. pipeline('object-detection', 'model-name') — without writing any low-level ONNX code yourself. This is THE library the problem statement is implicitly pointing at when it mentions 'Transformers.js and ONNX Runtime Web.'

WebGPU vs WASM

CONCEPT — WebGPU vs WebAssembly (WASM): Two ways the browser can execute the local model's math. WASM runs on the CPU and works on virtually every modern browser (universal, but slower). WebGPU offloads computation to the GPU (much faster) but browser/OS/driver support is still inconsistent as of 2026. Correct engineering approach: try WebGPU first, and automatically fall back to WASM if WebGPU is unavailable — never assume WebGPU will be there.

Vision Model / Object Detection vs Full Scene Understanding

CONCEPT — Object Detection: A model that finds WHERE specific things are in an image and draws a bounding box around each (e.g. 'a face is located at these pixel coordinates'). This is different from — and much lighter/faster than — 'scene understanding,' where a model tries to describe or reason about an entire image holistically (what GPT-4-Vision-class models do). Our local browser model should only do the lightweight job: detect and locate specific known categories (faces, text regions matching PII patterns) — not attempt full scene understanding, which is far too heavy to run smoothly in a browser tab.

OCR (Optical Character Recognition)

CONCEPT — OCR: Technology that reads text out of an image (pixels) and turns it into actual machine-readable text/strings. We need this because some sensitive text (like a card number typed into a canvas-rendered app, or text baked into an image/screenshot) has no DOM representation to read directly — OCR is the fallback path for those cases.

PII (Personally Identifiable Information)

CONCEPT — PII: Any data that could identify a specific person or that is sensitive in nature — names, emails, phone numbers, card numbers, government ID numbers, addresses, faces. Not all PII is equally sensitive (see Tiered Sensitivity Model, Section 8).

Redaction

CONCEPT — Redaction: The act of removing or hiding sensitive content before it is shared further — in our case, drawing solid, opaque boxes over sensitive screen regions before compositing the final image that gets sent to the server, or replacing a sensitive VALUE with a generic placeholder/token while keeping its TYPE information (see Semantic Redaction Tokens, Section 9).

LLM / VLM (Large Language Model / Vision-Language Model)

CONCEPT — LLM vs VLM: An LLM reads and generates TEXT only. A VLM (vision-language model) can additionally accept an IMAGE as input alongside text and reason about both together. In our system, the server's reasoning step is mostly a plain LLM call over structured TEXT (DOM summary + semantic tokens) — we only need a VLM if we decide to also send a redacted IMAGE for the server to look at directly, which is optional (see Section 3.3).

Structured Output / Function-calling style JSON

CONCEPT — Structured Output: Instead of letting an LLM reply with free-flowing prose, we constrain it (via prompt instructions and response validation) to always reply in a fixed JSON shape, e.g. {"action": "click", "target": "#submit-btn"}. This makes the LLM's output directly and safely executable by code, instead of needing to parse natural language.

Session / Task State

CONCEPT — Session State: Multi-step tasks (search -> select -> fill form -> pay -> confirm) require the server to remember what has already happened. We track this with a session_id that both client and server pass back and forth, with the server keeping a small in-memory (or Redis) record of what step the task is on.

Groq / LPU

CONCEPT — Groq: An AI inference provider that runs open-source LLMs (Llama, GPT-OSS, etc.) on custom chips called LPUs (Language Processing Units), which are exceptionally fast for LLM inference — hundreds of tokens per second, versus tens to low-hundreds on typical GPU-based cloud APIs. We use Groq specifically for the reasoning/decision LLM call because our end-to-end latency metric directly rewards a fast reasoning step.

3. System Architecture — Full Detail

3.1 High-level diagram (described)

Browser tab (real webpage) → Content script captures DOM snapshot + screenshot → Local detection pipeline (DOM heuristics + ONNX vision model + OCR) finds sensitive regions → Redaction engine draws solid boxes / generates semantic tokens → Sanitized payload assembled → Sent over HTTPS to FastAPI backend → Backend calls Groq LLM with task context + sanitized payload → LLM returns a structured action → Backend validates and forwards action to extension → Content script executes the action on the real (unredacted) page → Loop repeats with an updated screen state until task is marked complete.

3.2 Client — Browser Extension (component-by-component)

Component	Responsibility	Key implementation notes
manifest.json (Manifest V3)	Declares permissions, content scripts, background service worker, and extension popup/side-panel UI	Needs 'activeTab', 'scripting', and host permissions for the demo sandbox site; keep permissions minimal and explicit for the pitch (a security-focused tool should not over-request permissions).
Capture module	Grabs a snapshot of the visible tab (pixels) plus a DOM snapshot (structure) at the moment of analysis	Use chrome.tabs.captureVisibleTab for pixels; use a content script querying document.body for DOM/text/attributes for structure. Capture both together so they can be cross-referenced.
Local inference engine	Loads and runs the ONNX vision model + OCR model inside the browser via Transformers.js	Initialize once on extension startup (not on first task) to avoid cold-start lag during the actual demo. Detect WebGPU availability; fall back to WASM automatically.
DOM heuristics module	Reads field types, autocomplete attributes, aria-labels, and nearby text to flag likely-sensitive fields with zero inference cost	This is checked FIRST, before running the vision model, since it's near-instant and covers most standard HTML forms.
PII pattern matcher	Regex/pattern checks over any extracted text (from DOM values or OCR output) for emails, phone numbers, card numbers, Aadhaar-like numbers, IFSC codes, etc.	Exact pattern list must be finalized and written down before coding (see Section 10 checklist).
Redaction engine	Draws solid, standardized-size occlusion boxes on a over detected sensitive regions; generates semantic placeholder tokens for the structured payload	Never uses blur (reversible); box sizes come from a small fixed set of size categories, not exact pixel-fit, to avoid leaking information through box shape.
Change/diff detector	Uses MutationObserver to detect when the DOM changes meaningfully, to avoid re-running full analysis on every tiny change	Debounce rapid mutations (e.g. wait 200-300ms of quiet) before re-triggering analysis — protects the resource-utilization score.
Transport module	Sends the sanitized payload to the backend and receives the action instruction back	Simple fetch() POST to the FastAPI endpoint is enough; WebSocket only if the team wants a more 'live' feeling demo and has spare time.
Action executor	Receives a validated action object and performs it on the REAL page (click, set value, scroll, wait)	Runs entirely inside the content script since it needs direct DOM access; must check the action against the risk-tier list (Section 9) before auto-executing.
Demo/audit UI (side panel or popup)	Shows the live split-screen (real vs sanitized) view and the running audit log	This is Dev 5's primary surface — see Section 12.

3.3 Server — Backend (component-by-component)

Component	Responsibility	Key implementation notes
FastAPI app	Exposes the /analyze (or similarly named) endpoint that receives sanitized payloads and returns actions	Use Pydantic models for both incoming payload and outgoing action so shapes are enforced automatically.
Session manager	Tracks task/session state across multiple steps of one task	Simple in-memory dict keyed by session_id is enough for a 3-day build; Redis only if time allows and multi-instance demo is needed (unlikely).
Reasoning LLM client (Groq)	Sends the task context + sanitized DOM summary + semantic tokens to a Groq-hosted LLM (e.g. Llama 3.3 70B or GPT-OSS-20B/120B) and requests a structured action back	This is the primary 'thinking' step and the main lever on end-to-end latency — Groq's speed is the direct payoff here.
Optional VLM path	If structured text context genuinely isn't enough, send the redacted (already-safe) image to a vision-capable model (e.g. Llama 3.2 11B Vision on Groq) as a secondary check	Treat this as optional/fallback, not the default path — keeps latency down and avoids depending on Groq's more limited vision offering for the core loop.
Offline/self-hosted fallback path	Same model family (Llama, etc.) served locally via Ollama or vLLM, swappable behind the identical API interface used for Groq	This is what lets you honestly tell judges: 'in a real ISRO deployment, this same code points at an internal, air-gapped model instead of Groq' — a direct answer to the actual motivation in Section 1.3.
Action validator	Validates the LLM's raw JSON output against a strict Pydantic schema before it's ever sent back to the client	Reject/retry on malformed output rather than forwarding anything unvalidated to the extension — this is a safety-critical checkpoint.
Audit logger	Records every step: what was received, what was decided, and why (confidence scores, redaction decisions)	Append-only JSON lines file or SQLite table is enough.

Groq reality-check (important): Groq's vision offering is a limited preview-tier model, not their core strength — their real advantage is blazing-fast TEXT/LLM inference. Use Groq for the fast reasoning/decision LLM call. Do NOT build your core pipeline's accuracy around Groq vision; keep vision work client-side (detection only) and let the server reason mostly over structured text.

3.4 Why the boundary is drawn exactly here

The privacy guarantee of this entire system rests on ONE architectural fact: the redaction step happens BEFORE the network request is constructed, inside the browser, on the user's own device. Everything to the left of that line (capture, detection, redaction) never has to be trusted by anyone outside the user's machine. Everything to the right of that line (transport, server, LLM) only ever sees data that has already been sanitized. Keep this boundary crisp in your code structure — it should be obvious, even to someone skimming your codebase, exactly which function is the last one to touch raw/unredacted data before anything is serialized for network transport.

4. Data Contracts — Exact JSON Schemas

Agree on these shapes BEFORE writing integration code. Every field below should be treated as fixed unless the whole team agrees to change it — this is what prevents Day-3 integration breakage.

4.1 Client -> Server payload

```
{
  "session_id": "string (uuid, stable across one full task)",
  "task_instruction": "string, e.g. 'book this event ticket'",
  "step_number": 3,
  "dom_summary": {
    "url": "string",
    "elements": [
      {
        "element_id": "string (stable selector or generated id)",
        "tag": "input | button | select | textarea | div ...",
        "role": "textbox | button | checkbox ...",
        "label_text": "string or null",
        "is_sensitive": true,
        "sensitivity_tier": "1 | 2 | 3",
        "sensitivity_type": "PASSWORD | CARD_NUMBER | EMAIL | NAME | AMOUNT | UNKNOWN",
        "semantic_token": "[CARD_NUMBER] or null if not sensitive",
        "bounding_box": {"x": 120, "y": 340, "w": 220, "h": 32}
      }
    ]
  },
  "redacted_image_base64": "string (PNG/JPEG, solid-box redacted; OPTIONAL if VLM path unused)",
  "detection_confidence_notes": [
    {"element_id": "string", "confidence": 0.92, "method": "dom_heuristic | vision_model | ocr_regex"}
  ]
}
```

4.2 Server -> Client action response

```
{
  "session_id": "string (echoed back)",
  "step_number": 4,
  "action": {
    "type": "click | type | scroll | wait | ask_user_confirmation | task_complete | task_failed",
    "target_element_id": "string, matches an element_id from dom_summary, or null",
    "value": "string or null (text to type, only for type actions; never a real sensitive value)",
    "risk_tier": "safe | risky",
    "reasoning_short": "string, 1 short sentence, shown in the audit log UI"
  },
  "confidence": 0.87
}
```

4.3 Field-level rules that must be enforced in code

- The server must NEVER receive a real sensitive value in any field — only semantic_token placeholders or fully redacted images. If a bug ever puts a real password into this payload, that is a critical failure, not a minor bug — add an assertion/guard on the client before every network call.
- risk_tier = 'risky' actions (submit, pay, delete-like actions) must always require local confirmation before execution, regardless of the LLM's own confidence score.
- session_id must be generated once per task and reused for every step of that task — a new session_id per step will break multi-step continuity.
- sensitivity_tier and sensitivity_type follow the Tiered Sensitivity Model defined in Section 8.1 — do not invent ad-hoc categories mid-build.

5. Step-by-Step Workflow — What Happens, In Order

1. **User invokes the agent** with a plain-language task instruction (e.g. via the extension popup: 'book this ticket').
2. **Extension captures** the current visible tab (pixels) and the DOM tree (structure) at the same moment.
3. **DOM heuristics run first** (near-zero cost): every input/textarea/button is checked for type, autocomplete, label text, and nearby text to flag likely-sensitive fields.
4. **Local vision model + OCR run next**, only over regions the DOM pass could not classify confidently (e.g. canvas-rendered content, images, unlabeled fields) — this keeps inference cost down.
5. **Each detected element is assigned a sensitivity tier** (1/2/3) and, if sensitive, a semantic_token placeholder is generated.
6. **Redaction engine draws solid, standardized-size boxes** on a canvas copy of the screenshot for any element tagged Tier 1 (and Tier 2 if policy says so).
7. **Payload is assembled** exactly per the Section 4.1 schema and sent via HTTPS POST to the backend.
8. **Backend validates the payload shape**, updates/loads the session state, and constructs a prompt for the reasoning LLM containing the task instruction, DOM summary, and prior step history.
9. **Groq LLM returns a structured action** matching the Section 4.2 schema; backend validates it against the Pydantic schema and rejects/retries on malformed output.
10. **Backend logs the step** to the audit log (what was sent, what was decided, confidence) and returns the action to the extension.
11. **Extension checks the action's risk_tier**: safe actions execute immediately; risky actions show a one-click local confirmation prompt first.
12. **Action executor performs the action** on the REAL, unredacted page (e.g. actually clicking the real submit button, typing the user's real card number that was never sent to the server).
13. **Loop repeats**: extension re-captures the updated screen state and sends the next step, until the LLM returns 'task_complete' or 'task_failed', or the retry cap is hit.

6. Tech Stack — Every Component, Justified

6.1 Client stack

Layer	Technology	Justification
Extension scaffold	Manifest V3 (Chrome/Edge primary; Firefox via browser_specific_settings key)	Required standard for Chrome/Edge; Firefox support needs only small manifest additions, not a rewrite.
Local inference runtime	Transformers.js (built on ONNX Runtime Web)	Directly matches the problem statement's suggested stack (WebGPU/WebAssembly + ONNX Runtime Web); mature pipeline API means less boilerplate.
Local vision model (face/region detection)	A small ONNX-exportable detector, e.g. a lightweight face detector (see Section 7 for exact candidates)	Must be small enough for real-time in-browser inference — full scene-understanding ViTs are far too heavy for this constraint.
OCR	Tesseract.js (or a distilled ONNX OCR model via Transformers.js)	Needed as fallback for text baked into images/canvas where DOM has no representation.
DOM/heuristics	Vanilla JavaScript (no framework needed)	This layer is simple attribute/text inspection — a framework adds overhead without benefit here.
Redaction rendering	HTML5 Canvas API	Native browser API, no dependency needed, full control over solid-box drawing.
Change detection	MutationObserver (native browser API)	Native, zero-dependency way to detect meaningful DOM changes and avoid wasted re-analysis.
Extension UI (popup/side panel)	Plain HTML/CSS/JS or a minimal framework (e.g. Preact) if the team is faster with it	Keep this simple — most of the 'wow' comes from what it SHOWS (split-screen, audit log), not the framework used to build it.

6.2 Transport

Layer	Choice	Justification
Protocol	REST over HTTPS (JSON body)	Fastest to build correctly in 3 days; a WebSocket adds real-time feel but also adds failure modes to debug under time pressure.
Payload format	JSON, matching Section 4 schemas exactly	Structured, versioned, and directly maps to Pydantic validation on the server.

6.3 Server stack

Layer	Technology	Justification
Web framework	FastAPI (Python)	Async-friendly, minimal boilerplate, automatic request/response validation via Pydantic, fast to stand up an endpoint.

Layer	Technology	Justification
Reasoning LLM provider	Groq API — Llama 3.3 70B or GPT-OSS-20B/120B (text-only reasoning)	Groq's LPU hardware delivers roughly 300-800 tokens/second versus 50-100 on typical GPU-based cloud APIs — this directly and measurably improves the end-to-end latency evaluation metric. Free tier (30 requests/min, ~1,000-14,400 requests/day depending on model) is comfortably enough for a live demo loop.
Optional VLM (fallback only)	Llama 3.2 11B Vision via Groq	Cheapest multimodal option on Groq if you decide the reasoning step also needs to see the redacted image directly; treat as optional, not default.
Offline/self-hosted fallback	Ollama or vLLM running the same Llama family locally, behind an identical API interface	This is your direct, credible answer to ISRO's real air-gapped-deployment motivation (Section 1.3) — swap one environment variable/base URL, no code rewrite.
Schema validation	Pydantic v2 models for both incoming payload and outgoing action	Enforces the Section 4 contracts automatically; rejects malformed LLM output before it reaches the client.
Session state	In-memory Python dict keyed by session_id (Redis optional upgrade if time allows)	Simplicity wins for a 3-day build; Redis adds operational overhead with little payoff at this scale.
Audit log storage	Local JSON-lines file or SQLite	Zero infrastructure setup, sufficient for a demo-scale audit trail.

7. Model & Library Shortlist

Concrete starting candidates so the team can begin downloading/testing on Day 0 instead of researching from scratch. Benchmark all of these on your own hardware before committing (see Section 10).

Need	Candidate(s)	Notes
In-browser ML runtime	Transformers.js (npm: @huggingface/transformers, formerly @xenova/transformers)	Uses ONNX Runtime under the hood; supports both WASM and WebGPU backends automatically; pipeline API covers object-detection, image-classification, and more.
Face / sensitive-region detection	A lightweight, purpose-built face detector such as a 'tiny face detector' model (compact, sub-1MB-to-few-MB class of model), or a small ONNX object-detection export browsable under Hugging Face's transformers.js-compatible model listings	Prioritize model SIZE and inference SPEED over maximum accuracy — this is a detection task, not a scene-understanding task. Confirm exact model choice via your own Day-0 benchmark (Section 10) since exact best-fit varies by what's currently published and license-clear.
General object/region detection (fallback)	DETR-based object detection models available in ONNX format for Transformers.js (e.g. under the Xenova/transformers.js and onnx-community Hugging Face orgs)	Heavier than a dedicated face detector (tens of MB) — use only if you need general-purpose region detection beyond faces, and budget extra load time for it.
OCR	Tesseract.js	Mature, MIT-licensed, well-documented, runs fully client-side; the safe default choice for browser OCR.
Reasoning LLM (server)	Llama 3.3 70B or GPT-OSS-20B/120B via Groq API	Free tier is generous enough for demo-scale traffic; OpenAI-compatible SDK usage means minimal integration code.
Vision-capable LLM (optional, server)	Llama 3.2 11B Vision via Groq	Cheapest multimodal option on Groq; treat as a fallback/secondary path, not the default reasoning path.
Offline/self-hosted LLM runner	Ollama or vLLM	Both can serve the same open-weight model families behind an OpenAI-compatible API, making the swap from Groq nearly a one-line change.

Action item for Dev 5 (Section 10 checklist): Before Day 1 ends, confirm exact model file(s) chosen for face/region detection and OCR are (a) available in ONNX format compatible with Transformers.js, (b) under a permissive license (Apache 2.0 / MIT preferred), and (c) benchmarked for load time + inference speed on at least two different laptops, including whichever machine will be used for the live demo.

8. Ideation Loopholes, Deep-Dive Fixes, and Design Decisions

8.1 The Tiered Sensitivity Model (core design decision — use this everywhere)

The single hardest problem in this project is that 'sensitive' is not a fixed label — the same data can be harmless in one context and critical in another (a name on a public forum vs. a name on a medical form). A binary sensitive/not-sensitive flag cannot capture this. Instead, use three tiers consistently across every module:

Tier	Definition	Examples	Redaction policy
Tier 1	Always sensitive, regardless of context	Passwords, card numbers, SSN/Aadhaar-like numbers, OTPs	Always hard-redact (solid box + semantic token); bias detection toward high recall even at the cost of precision
Tier 2	Context-dependent sensitivity	Names, raw amounts, addresses, phone numbers	Redact by default; can be shown to the LLM as a semantic token with type preserved (e.g. [AMOUNT: range]) so task utility is preserved
Tier 3	Task-relevant, generally safe	Button labels, page titles, non-personal form structure	Not redacted; sent as-is to preserve full utility for the reasoning LLM

State this bounded, tiered claim explicitly in the pitch rather than claiming to have 'solved' contextual sensitivity in general — judges respect an honest, well-reasoned boundary far more than an unbounded claim that breaks under a single probing question.

8.2 Full loophole-to-fix table

Loophole	Why it breaks things	Fix / design decision
Contextual sensitivity	Same data type carries different risk depending on context; static rules alone can't tell.	Apply the Tiered Sensitivity Model (8.1) uniformly. Combine DOM-context signals (nearby labels, page type) with content pattern-matching rather than relying on field name alone.
Redaction breaks task utility	Over-redacting numbers/text can make the reasoning LLM unable to figure out the next correct action (e.g. can't see a total to decide the next click).	Semantic placeholder tokens instead of blind blackout: mask the VALUE, preserve the TYPE/structure, e.g. [AMOUNT: $XX.XX range], [CARD_NUMBER]. This is the single most important design decision in the whole project — revisit Section 4.1's semantic_token field.
False sense of privacy / redaction leakage	Blur is mathematically reversible for small radii; a redaction box exactly sized to a 16-digit number leaks its own meaning; raw DOM attributes (value="") can leak data even when the visible image is redacted.	Use solid occlusion, never blur, for anything security-critical. Standardize redaction box sizes into a small fixed set of categories (short/medium/long) rather than exact pixel-fit boxes. Audit EVERY channel in the outgoing payload — image, DOM summary, any attribute values — not just the screenshot.
Local model accuracy ceiling	A lightweight in-browser model cannot match a large cloud VLM's full scene understanding.	Explicitly scope the local model to narrow region DETECTION (bounding boxes for known categories), never attempt full scene comprehension client-side — that reasoning happens server-side, over already-safe data.
Latency vs accuracy trade-off	Running inference on every frame or every DOM mutation causes visible lag, especially without WebGPU.	Event-triggered inference only: run on explicit user invocation or after a debounced (200-300ms quiet period) DOM mutation — never continuous polling.

Loophole	Why it breaks things	Fix / design decision
Cross-origin / iframe blind spots	Payment widgets (Stripe/Razorpay-style checkouts) are frequently loaded in cross-origin iframes that browser extensions cannot read by design — this is a deliberate PCI-compliance security boundary, not a bug.	Acknowledge this as a known, honest architectural boundary in the pitch. Do not attempt to bypass cross-origin restrictions — that would itself be a security red flag to judges.
Adversarial / edge screen states	Canvas-rendered apps (e.g. Figma, Google Docs-style editors), CAPTCHAs, and video have no normal DOM text, so DOM heuristics fail completely and pure pixel/OCR detection becomes mandatory (slower, less reliable).	Explicitly scope the demo to standard HTML/DOM-based sites and state this boundary rather than implying full generality.
Server-side trust assumption	Nothing currently stops a compromised or malicious server from simply requesting raw, unredacted data — the trust boundary is enforced client-side only, with no verification the client's redaction was honest or complete.	This is a genuinely open research problem (akin to ongoing debates around client-side scanning) — you are not expected to solve it. State it plainly: 'a production version would need attestation or server-side verification of payload provenance.' Naming this unprompted signals real technical maturity.
Asymmetric false-positive / false-negative cost	Missing a real piece of Tier-1 PII (false negative) is a genuine privacy failure; over-redacting a harmless field (false positive) is merely an annoyance — but a naive precision/recall optimization treats both errors as equally costly.	Deliberately bias your detection threshold toward higher RECALL on Tier-1 categories, even if it costs precision. State this design choice explicitly — it maps directly to how recall and precision are separately weighted in the rubric.
Risky action execution	A server-issued instruction, based on a possibly-incomplete redacted context, could tell the agent to submit a payment, delete something, or take another irreversible action by mistake.	Risk-tiered action confirmation (see Section 4.2's risk_tier field): auto-execute safe actions (scroll, read, navigate); require one explicit local confirmation click for risky actions (submit, pay, delete).
Statefulness across multi-step tasks	Realistic tasks are rarely single-shot (search -> select -> fill -> pay -> confirm); if each step is analyzed independently with no memory, the reasoning LLM keeps losing track of progress.	Maintain lightweight session/task state (session_id + current step + fields already filled) passed with every request, per the Section 4 schema.
Error recovery / stuck states	An action that fails (element not found, an unexpected popup appears, page navigated away) can leave the demo visibly frozen in front of judges if there's no recovery path.	Implement a timeout + retry + re-analyze-with-updated-screen loop, with a hard maximum retry count so it can never infinite-loop live.
Dynamic / SPA pages	Modern React/Vue-style apps re-render the DOM constantly without full page reloads; reading the DOM only once on page load produces stale or missing element data.	Drive re-analysis off MutationObserver events (debounced) rather than a single one-time read on page load.
WebGPU availability isn't universal	WebGPU support varies across browsers, operating systems, and GPU drivers as of 2026 — a hard dependency on it risks an outright failure on the judges' demo machine.	Build with automatic graceful fallback to WASM/CPU (this is the default, well-supported path in ONNX Runtime Web / Transformers.js). Explicitly test on at least two different laptops before demo day, including whichever machine will actually be used to present.

Loophole	Why it breaks things	Fix / design decision
Cold-start latency	Loading a model into the browser for the first time (download + WASM/WebGPU initialization) can take several seconds, creating a laggy first impression.	Preload the model on extension install/startup rather than on first task invocation, and show a clear, honest loading indicator rather than a silent freeze.
Scope creep — trying to support 'any' website	Attempting full generality across the open web means fighting endless real-world edge cases and burning all 3 days on robustness instead of the core concept.	Demo on one or two self-built, fully-controlled sandbox pages (a fake login/checkout form) where you own the entire DOM structure. This is standard, accepted hackathon practice — judges evaluate whether the CONCEPT works cleanly, not whether it generalizes to the entire internet.
Silent task failure	If the agent doesn't complete a task and there is no visible reasoning trail, it just looks broken to a judge, with no insight into why.	Build a visible log/console panel (Dev 5's audit-log UI) showing each detection -> redaction -> server-decision -> execution step in plain language. This turns a possible failure into a transparent, still-impressive 'you can see exactly what it was thinking' moment.

9. USPs / Standout Features — Ranked and Explained

This problem statement has a fairly fixed architecture shape, so differentiation comes from executing the hard trade-offs visibly and well, not from inventing an unrelated new idea. USPs below are ranked by impact-to-effort so must-haves get built first and stretch items can be cut safely if time runs short.

Priority	USP	What it is	Why it matters
Must-have	Live split-screen "what the server sees" view	Render the real live tab and the exact sanitized payload being transmitted, side by side, updating in real time.	Cheapest possible build (you already have the redacted image — just render both) with the single highest demo impact: it makes an abstract privacy claim visually undeniable in about five seconds.
Must-have	Hybrid DOM + Vision detection	Combine free DOM signals (type=password, autocomplete, aria-label, nearby text) with the vision model only as a fallback for canvas/non-DOM content.	Higher precision, faster, and cheaper to build than a vision-only approach; defensible in Q&A; as 'we used the right tool for each part of the problem' rather than 'we used vision everywhere because the title says vision.'
Should-have	Semantic redaction tokens	Replace sensitive VALUES with typed placeholders ([CARD_NUMBER], [AMOUNT: range]) instead of blind blackout, preserving structure for the reasoning LLM.	Directly improves both visual-context-accuracy AND redaction-precision scoring simultaneously — most competing teams will trade one for the other.
Should-have	Redaction audit log	A structured, running log: 'Field #3 detected as EMAIL (confidence 0.91) -> redacted -> Field #5 detected as AMOUNT (confidence 0.40) -> sent as approximate range.'	Near-free to build (just structured logging) but turns the system from an opaque black box into something explainable — high value for a privacy-focused pitch.
Nice-to-have	On-device caching / diffing	Cache the last analyzed frame; only re-process regions that actually changed, using MutationObserver-based diffing.	Directly and measurably improves the client-side resource-utilization score (20% weight) — most hackathon teams won't bother with this level of discipline.
Nice-to-have	Risk-tiered action confirmation	Auto-execute safe actions (scroll, read, navigate); require one-click local confirmation for risky ones (submit, pay, delete).	Differentiates on agent SAFETY, not just privacy — most competitors will focus only on redaction and forget that a misfired action is its own real risk.
Stretch	Confidence-gated tiered redaction	Per-element confidence score drives behavior: high confidence -> hard redact; medium confidence -> redact but flag to the LLM as an uncertain, non-actionable region.	Targets the recall/precision metric directly and lets you explain your threshold-tuning logic live if asked — but requires careful calibration time, so treat as a stretch goal.
Stretch	User override / manual correction	Let the user click a redacted box to reveal 'actually this is fine to send,' or click an unredacted element to say 'hide this too.'	Addresses a real, unavoidable gap in any automated PII system (false positives/negatives always exist) with a small UI feature — good maturity signal, low build cost, but cut first if time runs out.

10. Pending Research Checklist (Do Before / While Building)

1. Benchmark 2-3 candidate face/region-detection ONNX models for load time and inference speed on a normal laptop (not just a high-end dev machine).
2. Benchmark 2 OCR options (Tesseract.js vs. a distilled ONNX OCR model available via Transformers.js) the same way.
3. Check model licenses for every chosen weight file — prefer Apache 2.0 / MIT; some face-detection weights are research-only licensed and would be an awkward gap if a judge asks about deployability.
4. Write down the exact PII regex/pattern set upfront as a shared team doc: email, 10-digit phone, 16-digit card, Aadhaar-like 12-digit, IFSC codes, and any others relevant to your demo scenario — this is the literal spec your detection layer implements.
5. Test WebGPU support on the actual demo machine(s) today; confirm the WASM fallback path genuinely works end-to-end if WebGPU is unavailable — do not discover this the night before presenting.
6. Spend 20-30 minutes surveying existing browser-based PII-redaction OSS/academic projects, so you have an informed, specific 'how we differ' answer if a judge asks 'isn't this like X.'
7. Write the exact demo script end to end: task instruction, which fields exist, which get redacted and how, what the LLM should decide at each step, and what 'success' looks like on screen — this document doubles as your literal test case.

Non-research setup steps (do alongside the research above)

- Assign explicit roles (Section 12) before Day 1 begins — do not let role division happen organically mid-build.
- Create and share the one-page architecture/schema reference (Sections 3 and 4 of this doc effectively already are this) so integration on Day 3 doesn't break from mismatched assumptions.
- Decide a fallback/degradation plan now: what happens live if WebGPU fails, or if Groq rate-limits mid-demo (keep a backup API key and/or a cached fallback response ready).
- Rehearse the 3-5 minute spoken pitch narrative as its own scheduled activity on Day 3, separate from coding time.

11. Day-by-Day Build Plan (3 Days, Hour-Level)

Day 1 — Foundations

Time block	Focus	Owner(s)
Hour 0-1	Team sync: finalize PII pattern list, sensitivity tiers, JSON schemas (Section 4) as the shared contract; confirm demo sandbox site scope	Whole team
Hour 1-4	Extension scaffold (manifest.json, content script injection, screen+DOM capture); begin Transformers.js integration and WebGPU/WASM fallback test	Dev 1
Hour 1-4	DOM heuristics module (type/autocomplete/label detection) and PII regex layer skeleton	Dev 2
Hour 1-4	FastAPI skeleton, Pydantic schemas from Section 4, Groq API key setup and first raw test call	Dev 3
Hour 1-4	Build the demo sandbox site (fake login/checkout form) with full DOM control; scaffold the content-script action executor	Dev 4
Hour 1-4	Model/license research (Section 10 items 1-3); set up shared repo structure and README	Dev 5
Hour 4-8	Get local vision model loaded and producing bounding boxes on the sandbox page; validate WASM fallback works	Dev 1
Hour 4-8	Wire DOM heuristics + regex matcher together; start canvas-based solid-box redaction rendering	Dev 2
Hour 4-8	First working /analyze endpoint returning a hardcoded valid action, to unblock client integration testing	Dev 3
Hour 4-8	Wire capture -> transport -> (mocked) action executor loop end-to-end with dummy data	Dev 4
Hour 4-8	Finish PII pattern list + demo script draft (Section 10 items 4, 7); begin split-screen UI skeleton	Dev 5

Day 2 — Core Pipeline

Time block	Focus	Owner(s)
Hour 0-4	Integrate DOM heuristics as the first-pass filter before triggering vision model; add MutationObserver-based debounced re-triggering	Dev 1

Time block	Focus	Owner(s)
Hour 0-4	Semantic token generation logic; finalize tiered sensitivity classification per Section 8.1	Dev 2
Hour 0-4	Real Groq call replacing the hardcoded response; add action schema validation + retry-on-malformed-output	Dev 3
Hour 0-4	Real action executor logic (click/type/scroll) against the real sandbox page; add risk-tier gating with confirmation prompt	Dev 4
Hour 0-4	Audit log backend hookup (structured JSONL); continue split-screen UI build	Dev 5
Hour 4-8	Full end-to-end integration test #1: real screen -> real redaction -> real Groq call -> real action execution	Dev 1 + Dev 4 lead, others support
Hour 4-8	Session/multi-step state wired in for a full multi-step task (not just single-shot)	Dev 3
Hour 4-8	On-device caching/diffing USP implementation (if on schedule)	Dev 1 + Dev 2
Hour 4-8	Offline/self-hosted fallback path (Ollama/vLLM) stood up behind the same interface	Dev 3
Hour 4-8	Finish split-screen + audit log UI polish; start pitch deck skeleton	Dev 5

Day 3 — Integration, Hardening, Pitch

Time block	Focus	Owner(s)
Hour 0-3	Full end-to-end run-through on the ACTUAL demo machine; fix WebGPU/WASM fallback issues discovered live	Dev 1
Hour 0-3	Error-recovery/retry loop hardening (timeout + max-retry cap); fix any redaction leakage found in testing	Dev 2 + Dev 4
Hour 0-3	Latency measurement pass; tune prompt/response size to Groq for speed	Dev 3
Hour 0-3	Record a backup demo video in case of live failure; finalize pitch deck with metrics table (Section 14)	Dev 5
Hour 3-6	Full team dry run of the live demo script from Section 10; fix any remaining rough edges	Whole team
Hour 6-8	Pitch narrative rehearsal (Section 15) as a dedicated activity, out loud, timed	Whole team

12. Team Division — 5 Developers, Full Responsibility Breakdown

Four members carry the heavy, interdependent technical work on the critical path. One member owns lower-stakes, high-visibility work that is intentionally decoupled from the critical path so nothing else blocks on it and it cannot break the core pipeline — but it remains highly visible in the final demo and judging outcome.

Role	Stakes	Responsibilities
Dev 1 — Extension & Local Vision Lead	HIGH	Manifest V3 scaffold; screen + DOM capture pipeline; Transformers.js + ONNX Runtime Web integration; local vision model wired in and producing bounding boxes; WebGPU-with-WASM-fallback tested on multiple machines; MutationObserver-based debounced re-triggering; on-device caching/diffing USP.
Dev 2 — Redaction & Hybrid Detection Lead	HIGH	DOM signal/heuristics module (type/autocomplete/label inspection); PII regex/pattern layer; tiered sensitivity classification (Section 8.1) implementation; canvas-based solid-box redaction with standardized sizes; semantic-token generation for the payload schema.
Dev 3 — Backend & LLM Integration Lead	HIGH	FastAPI service and Pydantic schemas; Groq API integration for the reasoning LLM; structured action-JSON validation and retry-on-malformed-output; session/multi-step task-state handling; Ollama/vLLM offline-fallback wiring behind the same interface.
Dev 4 — Action Executor & Integration Lead	HIGH	Content-script action executor (click/type/scroll on the real page); client<->server transport wiring; end-to-end loop integration and testing; risk-tiered confirmation logic; error-recovery/retry loop; demo sandbox site build.
Dev 5 — Demo, Docs & Polish (lower-stakes, still valuable)	LOWER / SUPPORTING	Live split-screen demo UI panel; redaction audit-log UI; README and architecture-doc upkeep; pitch deck creation; demo-script rehearsal support; model-license checks and benchmark data collection (Section 10); recording a backup demo video for live-failure contingency.

Dev 5's work is deliberately decoupled from the critical path — nothing else blocks on it, and it cannot break the core pipeline if something goes wrong. It is still highly visible in the demo (the split-screen view and audit log are often the most memorable part for judges), so it is low-stakes technically but high-value for the overall judging outcome.

13. Risk Register & Live-Demo Failure Plan

Risk	Likelihood	Impact	Mitigation
WebGPU unsupported on demo machine	Medium	High	Confirmed WASM fallback tested well before demo day; never assume WebGPU will be present.
Groq rate limit hit mid-demo	Low-Medium	High	Keep a backup API key ready; pre-cache one known-good response for the exact demo script as an emergency fallback.
Local model too slow / laggy live	Medium	Medium	Preload model on extension startup; choose the smallest viable model per Section 7/10 benchmarks.
Action executor clicks the wrong element	Medium	Medium	Risk-tiered confirmation for anything irreversible; thorough dry runs on the exact demo sandbox page.
Redaction misses a real PII value live	Medium	High (credibility)	Bias detection toward recall on Tier-1 categories; rehearse only on the pre-tested sandbox page, not an unfamiliar live site.
Live demo freezes entirely	Low	High	Backup demo video recorded in advance (Dev 5) as an explicit fallback to switch to without breaking pitch flow.
Judge asks about a loophole not yet mentioned	Medium	Medium	Section 8 table doubles as a rehearsed Q&A; reference — read it as a team before presenting.

14. Evaluation Metrics Recap & Design-Choice Mapping

Metric	Weight	Design choices that directly target it
Accuracy of visual context from screen	25%	Hybrid DOM+vision detection (Section 9); semantic tokens preserving structure (Section 8.1/9)
Recall & precision for PII detection	20%	Tiered sensitivity model (8.1); recall-biased thresholds on Tier-1 categories; confidence-gated redaction (stretch USP)
Precision of redaction	20%	Solid, standardized-size masks (never blur); full-payload leakage audit (Section 8.2)
Client-side resource utilization	20%	Event-triggered inference; DOM-first hybrid approach; on-device caching/diffing USP
Overall end-to-end latency	15%	Groq for fast reasoning LLM calls; keeping server-side vision optional rather than default

15. Pitch Narrative & Framing

Opening line: "Government and defense organizations increasingly want AI copilot tools, but can't risk sensitive operational data leaving their network to reach cloud AI providers. We built a way to get AI-agent assistance while guaranteeing sensitive data never crosses the boundary — demonstrated here on a browser, but architecturally applicable to any internal enterprise tool."

This framing shows you understood the real motivation behind the problem statement (Section 1.3), not just the literal feature list, and makes the solution sound broadly valuable rather than a narrow browser-extension toy.

Suggested pitch structure (3-5 minutes)

1. Open with the reframed motivation line above (10-15 seconds).
2. Show the live split-screen demo USP immediately — this is your strongest visual hook.
3. Narrate one full task loop out loud while it runs, pointing at the audit log as it updates.
4. State your two or three strongest design decisions explicitly (semantic tokens, tiered sensitivity, hybrid detection) and WHY, in one sentence each.
5. Proactively name one loophole you knowingly did not fully solve (e.g. server-side trust) — this signals maturity rather than being caught off guard by a judge's question.
6. Close by mapping your build back to the ISRO motivation: 'this same pipeline points at an internal offline model in production.'

16. Glossary of Terms

Term	Definition
Manifest V3	Current standard format for Chrome/Edge browser extensions defining permissions and script structure.
Content script	Extension JavaScript injected into a webpage; can read/modify that page's DOM.
DOM	The structured tree representation of a webpage's HTML that the browser builds and scripts can read/modify.
ONNX	A universal file format for trained ML models, allowing them to run across different runtimes/platforms, including browsers.
ONNX Runtime Web	The browser-compatible engine that executes .onnx model files via WASM or WebGPU.
Transformers.js	A JavaScript library providing a simple pipeline API to run pretrained ONNX models in-browser.
WebGPU	A browser API for GPU-accelerated computation; faster than WASM but not universally supported yet.
WASM (WebAssembly)	A universally-supported browser execution format; used as the CPU fallback for local inference.
OCR	Optical Character Recognition — extracting readable text from image pixels.
PII	Personally Identifiable Information — any data that could identify or is sensitive to a specific person.
Redaction	Hiding or removing sensitive content before further sharing (e.g. solid-box masking, semantic tokens).
LLM	Large Language Model — reads/generates text.
VLM	Vision-Language Model — can additionally accept and reason over images.
Structured output	Constraining an LLM's response to a fixed, machine-parseable JSON shape.
Session state	Server-side tracking of progress through a multi-step task, keyed by a session_id.
Groq / LPU	An AI inference provider using custom Language Processing Unit chips for very fast LLM inference.
MutationObserver	A native browser API that detects changes to the DOM, used to trigger re-analysis efficiently.
Risk tier (safe/risky)	Classification of an agent action by reversibility/consequence, gating whether it auto-executes or needs confirmation.

End of document. This is the canonical reference for the team — use it to generate build prompts for any module, resolve doubts about scope or design decisions, and onboard anyone picking up a piece of this project mid-hackathon.